PicoRV32 SoC

Peripheral Register Datasheet

Complete hardware register reference for UART · LED · Timer · TRNG

CPU	RV32I (PicoRV32)	**Clock**	100 MHz (10 ns / tick)
FPGA	Xilinx Spartan-7 XC7S50	**Board**	RealDigital Boolean Board
Bus	PicoRV32 Native Memory Bus	**App RAM**	8 KB @ 0x1000_0000
Baud Rate	115200 (UART)	**Boot ROM**	2 KB @ 0x0000_0000

Table of Contents

1 **System Memory Map** 2
2 **UART — 0x2000_0000** 2
2.1 UART_DATA (0x2000_0000) 2
2.2 UART_STATUS (0x2000_0004) 3
2.3 C Usage Examples 3
3 **LED — 0x3000_0000** 3
3.1 LED_REG (0x3000_0000) 3
3.2 C Usage Examples 4
4 **Hardware Timer — 0x4000_0000** 4
4.1 TIMER_CNT (0x4000_0000) 4
4.2 TIMER_CMP (0x4000_0004) 4
4.3 TIMER_CTRL (0x4000_0008) 5
4.4 C Usage Examples 5
5 **TRNG — 0x5000_0000** 5
5.1 TRNG_DATA (0x5000_0000) 5
5.2 TRNG_STATUS (0x5000_0004) 6
5.3 TRNG_CTRL (0x5000_0008) 6
5.4 TRNG_DIV (0x5000_000C) 7
5.5 C Usage Examples 7
6 **C Driver API Quick Reference** 7

1 System Memory Map

All peripherals are memory-mapped. The CPU accesses them using standard RISC-V load/store instructions. The upper nibble of the 32-bit address selects the peripheral (bits [31:28]).

Base Address	End Address	Size	Peripheral	Description
0x0000_0000	0x0000_07FF	2 KB	**Boot ROM**	Bootloader (bootloader.hex). R/W.
0x1000_0000	0x1000_1FFF	8 KB	**App RAM**	Application code + data. Written by bootloader at runtime.
0x2000_0000	0x2000_0007	8 B	**UART**	Serial port — TX/RX data and status.
0x3000_0000	0x3000_0003	4 B	**LED**	16-bit LED output register.
0x4000_0000	0x4000_000B	12 B	**Timer**	32-bit free-running counter + compare + control.
0x5000_0000	0x5000_000F	16 B	**TRNG**	True Random Number Generator (ring-oscillator).

***NOTE** — Unaligned or unmapped accesses return 0x00000000 and do not stall the bus.*

2 UART

An 8-N-1 UART with a fixed baud rate of 115,200 baud. The baud divisor is hard-coded to 868 clock cycles per bit (100 MHz ÷ 115,200). TX and RX share the DATA register address — direction is determined by a write vs. a read transaction.

2.1 UART_DATA

UART_DATA 0x2000_0000 **R/W**

Bit(s)	Field Name	Access	Reset
31:8	–	RO	0x000000
7:0	DATA[7:0]	R/W	0x00

Bit(s)	Field	Description
31:8	–	Reserved. Always reads 0. Writes ignored.
7:0	DATA[7:0]	WRITE: Loads byte into TX shift register and begins 8-N-1 transmission. Only valid when TX_BUSY = 0. READ: Returns the last received byte and clears RX_VALID (acknowledge).

2.2 UART_STATUS

UART_STATUS 0x2000_0004 **RO**

Bit(s)	Field Name	Access	Reset
31:2	—	RO	0
1	RX_VALID	RO	0
0	TX_BUSY	RO	0

Bit(s)	Field	Description
31:2	—	Reserved.
1	RX_VALID	Set to 1 when a complete byte has been received and is waiting in the RX buffer. Cleared automatically when UART_DATA is read.
0	TX_BUSY	Set to 1 while the TX shift register is actively transmitting. Must be 0 before writing a new byte to UART_DATA.

2.3 C Usage Examples

```
// Transmit one byte
while (REG_UART_STATUS & 0x01); // wait TX_BUSY = 0
REG_UART_DATA = 'A';

// Receive one byte (blocking)
while (!(REG_UART_STATUS & 0x02)); // wait RX_VALID = 1
char c = (char)(REG_UART_DATA & 0xFF);

// Non-blocking RX check
if (REG_UART_STATUS & 0x02) { char c = (char)(REG_UART_DATA & 0xFF); }

// Driver API
uart_putc('H'); // blocks until TX_BUSY clears, then sends
uart_puts("Hello!\n"); // sends string, auto-inserts \r before \n
char b = uart_getc(); // blocks until RX_VALID = 1
```

3 LED

Direct-mapped 16-bit output register. Each bit controls one physical LED on the Boolean board. Write 1 to turn ON, write 0 to turn OFF. Reads return the current register value.

3.1 LED_REG

LED_REG 0x3000_0000 **R/W**

Bit(s)	Field Name	Access	Reset
31:16	—	RO	0x0000
15:0	LED[15:0]	R/W	0x0000

Bit(s)	Field	Description
31:16	—	Reserved. Writes ignored, reads return 0.
15:0	LED[15:0]	Each bit directly drives one physical LED. 1 = ON, 0 = OFF. Writes are byte-lane granular: wstrb byte 0 controls LED[7:0], byte 1 controls LED[15:8]. Reads return the currently latched value.

3.2 C Usage Examples

```
REG_LEDS = 0xFFFF; // all ON
REG_LEDS = 0x0000; // all OFF
REG_LEDS = (1U << 7); // LED7 only
REG_LEDS ^= (1U << 3); // toggle LED3

// Driver API
led_set(0xAAAA); // alternating pattern
led_toggle(0x00FF); // toggle lower 8 LEDs
led_set_bit(5); // set LED5
led_clear_bit(5); // clear LED5
```

4 Hardware Timer

A 32-bit memory-mapped hardware timer. At reset the counter is enabled and free-runs at 100 MHz (10 ns per tick). When the counter reaches TIMER_CMP, it resets to zero, sets the MATCH flag, and asserts IRQ.

***NOTE** — Timing reference: 1 µs = 100 ticks | 1 ms = 100,000 ticks | 1 s = 100,000,000 ticks (0x05F5_E100)*

4.1 TIMER_CNT

TIMER_CNT 0x4000_0000 **RO**

Bit(s)	Field Name	Access	Reset
31:0	CNT[31:0]	RO	0x00000000

Bit(s)	Field	Description
31:0	CNT[31:0]	Current 32-bit counter value. Increments every clock cycle (10 ns at 100 MHz). Automatically resets to 0 on a compare match. Wraps at 0xFFFFFFFF if CMP = 0 (compare disabled).

4.2 TIMER_CMP

TIMER_CMP 0x4000_0004 **R/W**

Bit(s)	Field Name	Access	Reset
31:0	CMP[31:0]	R/W	0x05F5E100

Bit(s)	Field	Description
31:0	CMP[31:0]	Compare value. When CNT >= CMP and CMP != 0, the counter resets to 0, the MATCH flag is set, and IRQ is asserted. Default reset value = 100,000,000 (1 second at 100 MHz). Set CMP = 0 to disable compare and let the counter free-run indefinitely.

4.3 TIMER_CTRL

TIMER_CTRL 0x4000_0008 **R/W**

Bit(s)	Field Name	Access	Reset
31:3	—	RO	0
2	MATCH	RO	0
1	CLR_IRQ	W1C	0
0	EN	R/W	1

Bit(s)	Field	Description
31:3	—	Reserved.
2	MATCH	Set to 1 when CNT has reached CMP. Sticky — stays set until CLR_IRQ is written. Poll this bit in software to detect timer expiry.
1	CLR_IRQ	Write 1 to clear the MATCH flag and deassert the IRQ line. Self-clearing (always reads 0). Typically combined with EN in one write: TIMER_CTRL = 0x03.
0	EN	Timer enable. 1 = counter runs. 0 = counter frozen. Default is 1 (running) after reset.

4.4 C Usage Examples

```
// Read raw tick count
uint32_t now = REG_TIMER_CNT;

// Spin-wait 1 ms (disable compare first: set CMP = 0)
REG_TIMER_CMP = 0;
uint32_t t0 = REG_TIMER_CNT;
while ((REG_TIMER_CNT - t0) < 100000);

// Set compare for 500 ms, poll MATCH flag
REG_TIMER_CMP = 50000000;
REG_TIMER_CTRL = 0x03; // EN=1, clear prior match
while (!(REG_TIMER_CTRL & 0x04)); // wait MATCH (bit 2)
REG_TIMER_CTRL = 0x03; // clear MATCH

// Driver API
timer_init(); // CTRL = 0x01
uint32_t c = timer_get_cycles(); // reads CNT
timer_delay_cycles(100000); // spin-wait 1 ms
```

5 TRNG — True Random Number Generator

A hardware TRNG based on a bank of 16 ring oscillators with inverter chain lengths of 3, 5, 7, ... 33 stages. Output bits are XOR-combined across all oscillators, optionally de-biased using the von Neumann algorithm, and assembled into 32-bit output words. Two NIST SP 800-90B health tests run continuously: Repetition Count Test (RCT) and Adaptive Proportion Test (APT).

***NOTE** — Startup: Write TRNG_CTRL = 0x03 (EN=1, VN_EN=1), then wait for WARMUP_DONE (STATUS bit 4) before reading TRNG_DATA.*

5.1 TRNG_DATA

TRNG_DATA 0x5000_0000 **RO (destructive)**

Bit(s)	Field Name	Access	Reset
31:0	RDATA[31:0]	RO	0x00000000

Bit(s)	Field	Description
31:0	RDATA[31:0]	A 32-bit true random word. READING THIS REGISTER POPS (consumes) THE WORD — DATA_VALID is cleared after the read. If DATA_VALID = 0 when read, returns 0x00000000 and sets the UNDERFLOW sticky flag.

5.2 TRNG_STATUS

TRNG_STATUS 0x5000_0004 **RO**

Bit(s)	Field Name	Access	Reset
31:6	—	RO	0
5	UNDERFLOW	RO sticky	0
4	WARMUP_DONE	RO	0
3	APT_FAIL	RO sticky	0
2	RCT_FAIL	RO sticky	0
1	HEALTH_FAIL	RO sticky	0
0	DATA_VALID	RO	0

Bit(s)	Field	Description
31:6	—	Reserved.
5	UNDERFLOW	Set when TRNG_DATA was read while DATA_VALID = 0. Sticky — cleared by writing CLR (bit 3 of TRNG_CTRL).
4	WARMUP_DONE	Set after 1024 raw entropy bits have been discarded at startup. Must be 1 before reading TRNG_DATA for valid random output.
3	APT_FAIL	Adaptive Proportion Test failure (NIST SP 800-90B 4.4.2). Sticky. Indicates a bias or stuck-bit in the raw bitstream.
2	RCT_FAIL	Repetition Count Test failure (NIST SP 800-90B 4.4.1). Sticky. Indicates oscillators may be phase-locked or output frozen.
1	HEALTH_FAIL	Logical OR of RCT_FAIL and APT_FAIL. Check this bit as the overall health indicator.
0	DATA_VALID	Set to 1 when a fresh 32-bit random word is available in TRNG_DATA. Cleared when TRNG_DATA is read. Always poll this before reading.

5.3 TRNG_CTRL

TRNG_CTRL 0x5000_0008 **R/W**

Bit(s)	Field Name	Access	Reset
31:4	—	RO	0
3	CLR	W1C	0
2	IRQ_EN	R/W	0
1	VN_EN	R/W	1
0	EN	R/W	0

Bit(s)	Field	Description
31:4	—	Reserved.
3	CLR	Write 1 to: clear all sticky status flags (HEALTH_FAIL, RCT_FAIL, APT_FAIL, UNDERFLOW), flush the output register, and restart the warm-up sequence. Self-clearing (always reads 0).
2	IRQ_EN	Interrupt enable. When set, IRQ is asserted when DATA_VALID = 1 or HEALTH_FAIL = 1. Note: IRQ is not connected in current SoC version.
1	VN_EN	Von Neumann de-biasing enable. 1 = raw bit pairs are de-biased (10→1, 01→0, 00/11 discarded) to improve bit balance. Reduces throughput by ~50%. Default ON at reset.
0	EN	Ring oscillator enable. 0 = oscillators stopped (low power). Must be set to 1 to generate entropy. Default 0 at reset.

5.4 TRNG_DIV

TRNG_DIV 0x5000_000C **R/W**

Bit(s)	Field Name	Access	Reset
31:16	—	RO	0
15:0	DIV[15:0]	R/W	hw

Bit(s)	Field	Description
31:16	—	Reserved.
15:0	DIV[15:0]	Sample period = DIV + 1 clock cycles. Higher value = more jitter accumulated per raw bit = better entropy quality but lower throughput. DIV = 0 gives the fastest sampling rate (1 raw bit per clock cycle).

5.5 C Usage Examples

```
// Standard startup sequence
REG_TRNG_CTRL = 0x03; // EN=1, VN_EN=1
REG_TRNG_DIV = 0; // fastest sample rate
while (!(REG_TRNG_STATUS & 0x10)); // wait WARMUP_DONE (bit 4)

// Read one random word (safe)
while (!(REG_TRNG_STATUS & 0x01)); // wait DATA_VALID (bit 0)
uint32_t r = REG_TRNG_DATA; // pops word, clears DATA_VALID

// Check health before use
if (REG_TRNG_STATUS & 0x02) { // HEALTH_FAIL
REG_TRNG_CTRL = 0x0B; // CLR=1 + EN=1 + VN_EN=1
while (!(REG_TRNG_STATUS & 0x10)); // re-warm
}

// Driver API
```

```
trng_init(); // EN=1, VN_EN=1
while (!trng_is_warm()); // poll WARMUP_DONE
uint32_t r = trng_read(); // blocking pop
bool ok = trng_has_data(); // non-blocking DATA_VALID check
```

WARNING — Reading TRNG_DATA when DATA_VALID = 0 returns 0x00000000. This is NOT a random value. Always check DATA_VALID or use trng_read().

6 C Driver API Quick Reference

All driver functions are defined in app/. Include the corresponding header and ensure the .c file is compiled via the Makefile.

Module	Function Signature	Description
UART	`void uart_putc(char c)`	TX one byte — blocks until TX_BUSY = 0
UART	`void uart_puts(const char *s)`	TX string; auto-inserts \r before \n
UART	`bool uart_available(void)`	Returns true if RX_VALID = 1
UART	`char uart_getc(void)`	RX one byte — blocks until RX_VALID = 1
UART	`void uart_print_dec(uint32_t v)`	Print decimal number over UART
UART	`void uart_print_hex32(uint32_t v)`	Print "0x12345678" over UART
LED	`void led_set(uint16_t value)`	Write all 16 LEDs at once
LED	`void led_toggle(uint16_t mask)`	XOR mask onto current LED state
LED	`void led_set_bit(uint8_t bit)`	Turn on single LED (bit 0–15)
LED	`void led_clear_bit(uint8_t bit)`	Turn off single LED (bit 0–15)
Timer	`void timer_init(void)`	Enable timer: TIMER_CTRL = 0x01
Timer	`uint32_t timer_get_cycles(void)`	Read TIMER_CNT register
Timer	`void timer_delay_cycles(uint32_t n)`	Spin-wait for n clock ticks
TRNG	`void trng_init(void)`	Enable: TRNG_CTRL = 0x03 (EN + VN_EN)
TRNG	`bool trng_is_warm(void)`	Returns true when WARMUP_DONE (bit 4)
TRNG	`bool trng_has_data(void)`	Returns true when DATA_VALID (bit 0)
TRNG	`uint32_t trng_read(void)`	Blocking pop — waits DV=1, reads word